Lab 8 - Edge Detection

Name: Mohammed Feras Sawab

ID: 2240009066

In [1]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

imgFile = 'kid1.jpg'
```

Sobel Edge Detector

In [2]:
```
# 1. Load the real image

img = cv2.imread(imgFile)

# 2. Convert to Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 3. Apply Gaussian Blur to reduce noise
# (5, 5) is the kernel size; 0 means standard deviation is calculated from k

# ---------| You can try Sobel with Original Image, Gray scale and Blur
# ---------| Try these
# ---(1)---|  blurred = img
# ---(2)---|  blurred = gray

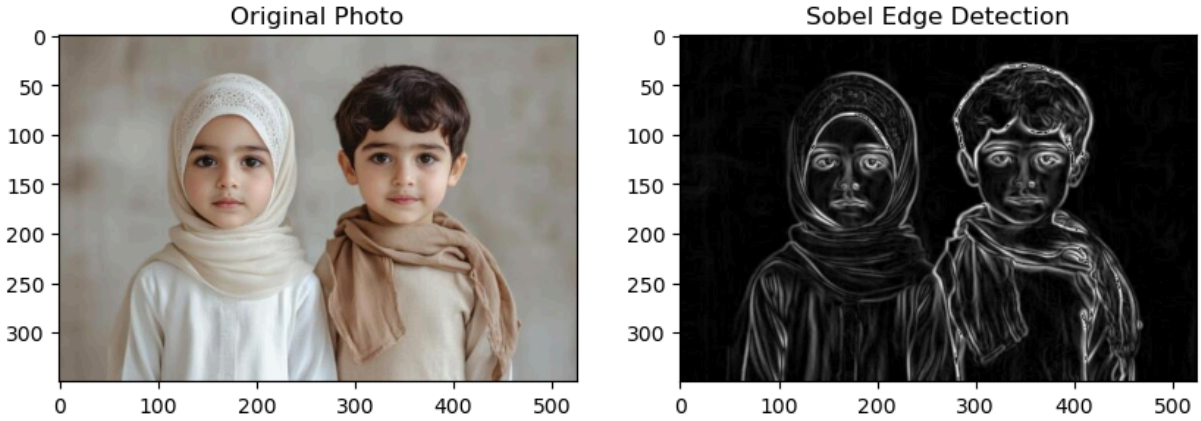
blurred = cv2.GaussianBlur(gray, (5, 5), 0) # blurred = img or gray   ## Che

# 4. Apply Sobel filter
# cv2.CV_64F handles the high precision needed for derivatives
sobelx = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)

# 5. Calculate Magnitude (Final Edge Map)
sobel_combined = cv2.magnitude(sobelx, sobely)

# 6. Normalize to 0-255 for display
sobel_final = np.uint8(np.absolute(sobel_combined))

# Visualization
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1), plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)), plt.
plt.subplot(1, 2, 2), plt.imshow(sobel_final, cmap='gray'), plt.title('Sobel
plt.show()
```



Robert Edge Detector

Comparing to Sobel

Roberts: Kernel Size --> 2x2 Sobel: Kernel Size --> 3x3

Roberts: Speed --> Faster Sobel: Speed --> Slower

Roberts: Noise Resistance --> Poor (High Senstivity) Sobel: Noise Resistnace --> Good (Built-in Smoothing)

Roberts: Edge Width --> Very thin/sharp Sobel: Edge Width --> Thicker

In [3]:
```
# 1. Load a real image in grayscale
image = cv2.imread(imgFile, 0) #cv2.IMREAD_GRAYSCALE)

# 2. Pre-process: Always blur for Roberts!
# Its 2x2 kernel is very sensitive to noise.
blurred = cv2.GaussianBlur(image, (5, 5), 0)

# 3. Define Roberts Cross kernels
kernel_x = np.array([[1, 0], [0, -1]], dtype=np.float64)
kernel_y = np.array([[0, 1], [-1, 0]], dtype=np.float64)

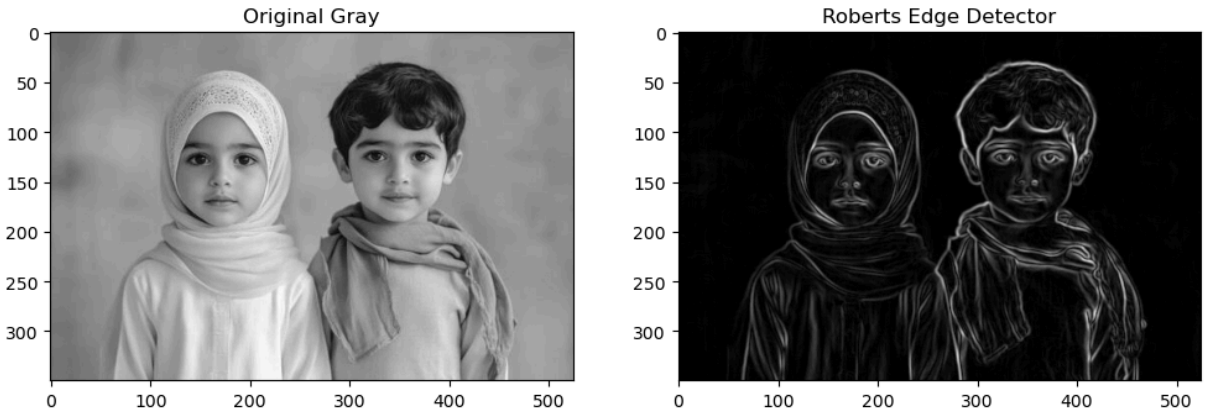
# 4. Apply kernels using convolution
roberts_x = cv2.filter2D(blurred, cv2.CV_64F, kernel_x)
roberts_y = cv2.filter2D(blurred, cv2.CV_64F, kernel_y)

# 5. Calculate Magnitude (Combine X and Y)
# Magnitude = sqrt(Gx^2 + Gy^2)
roberts_combined = cv2.magnitude(roberts_x, roberts_y)

# 6. Convert to uint8 for display
roberts_final = np.uint8(np.absolute(roberts_combined))

# Display
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1), plt.imshow(image, cmap='gray'), plt.title('Original Gr
```

```
plt.subplot(1, 2, 2), plt.imshow(roberts_final, cmap='gray'), plt.title('Rob
plt.show()
```



Prewitt Edge Detector

In [4]:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load a real image in grayscale
img = cv2.imread(imgFile)

# 2. Grayscale & Blur (Essential for real photos to reduce grain)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

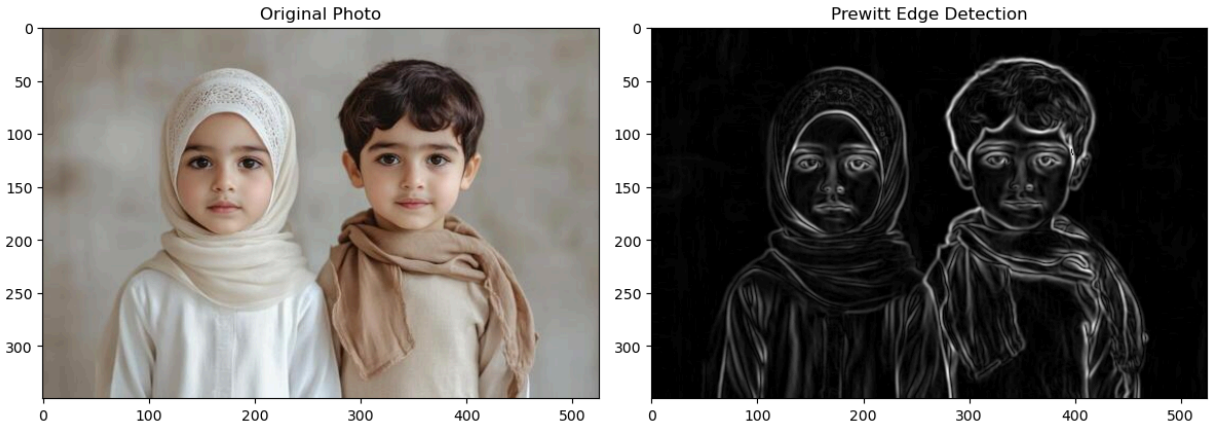
# 3. Define Prewitt Kernels manually
kernel_x = np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]], dtype=np.float64)
kernel_y = np.array([[-1, -1, -1], [0, 0, 0], [1, 1, 1]], dtype=np.float64)

# 4. Apply filters
prewitt_x = cv2.filter2D(blurred, cv2.CV_64F, kernel_x)
prewitt_y = cv2.filter2D(blurred, cv2.CV_64F, kernel_y)

# 5. Calculate Magnitude (Combine X and Y)
prewitt_combined = cv2.magnitude(prewitt_x, prewitt_y)

# 6. Normalize to 8-bit for display
prewitt_final = np.uint8(np.absolute(prewitt_combined))

# Visualizing results
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1), plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)), plt.
plt.subplot(1, 2, 2), plt.imshow(prewitt_final, cmap='gray'), plt.title('Pre
plt.tight_layout()
plt.show()
```



Canny Edge Detector

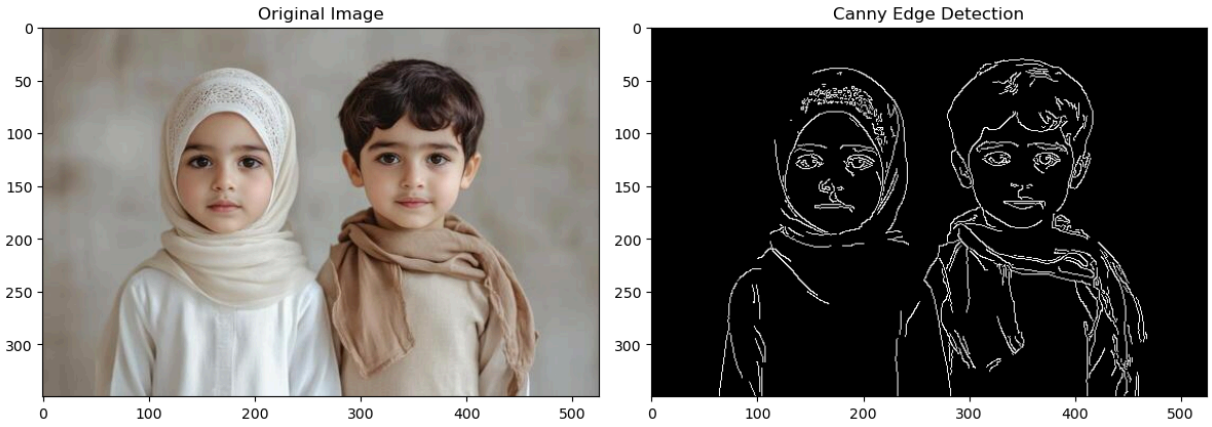
In [5]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load the real image
# Replace 'building.jpg' with your file path
img = cv2.imread(imgFile)


# 2. Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 3. Apply Canny Edge Detection
# Syntax: cv2.Canny(image, low_threshold, high_threshold)
# Tip: A 1:2 or 1:3 ratio for thresholds usually works best
canny_edges = cv2.Canny(gray, 100, 200)

# 4. Visualization
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1), plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)), plt.
plt.subplot(1, 2, 2), plt.imshow(canny_edges, cmap='gray'), plt.title('Canny
plt.tight_layout()
plt.show()
```



Multi-Channel Canny Edge Detector

In [6]:
```python
def best_color_edge_detector(image_path):
    # 1. Load the color image
    img = cv2.imread(image_path)
    if img is None:
        return "Image not found."

    # 2. Split into B, G, R channels
    channels = cv2.split(img)
    edge_maps = []

    for channel in channels:
        # 3. Pre-process each channel with Gaussian Blur to reduce noise
        blurred = cv2.GaussianBlur(channel, (5, 5), 0)

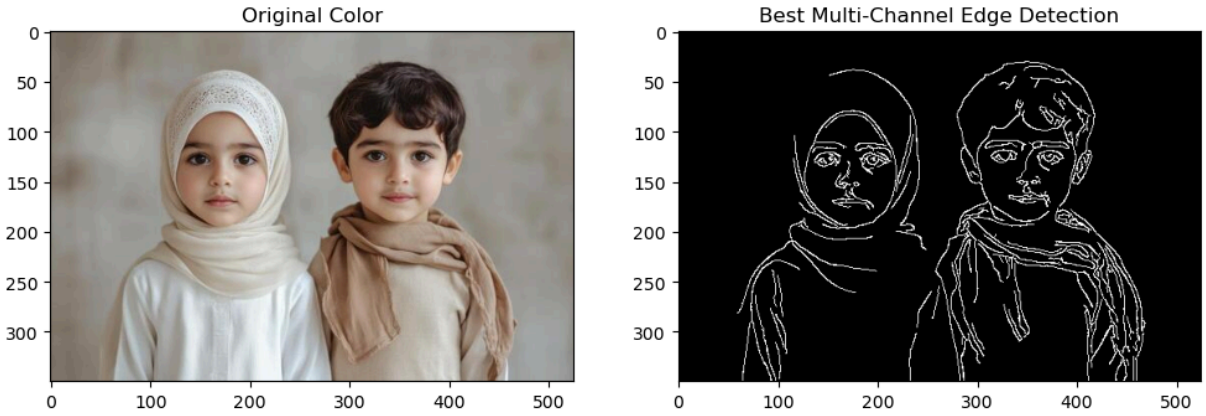
        # 4. Apply Canny to each channel
        # 50 and 150 are the low and high thresholds
        edges = cv2.Canny(blurred, 50, 150)
        edge_maps.append(edges)

    # 5. Merge edges from all channels using bitwise OR
    # This keeps any edge found in ANY of the three color layers
    combined_edges = cv2.bitwise_or(edge_maps[0], edge_maps[1])
    combined_edges = cv2.bitwise_or(combined_edges, edge_maps[2])

    return img, combined_edges

# Run the detector
original, edges = best_color_edge_detector(imgFile)

# Display results
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1), plt.imshow(cv2.cvtColor(original, cv2.COLOR_BGR2RGB)),
plt.subplot(1, 2, 2), plt.imshow(edges, cmap='gray'), plt.title('Best Multi-
plt.show()
```



LoG Filter

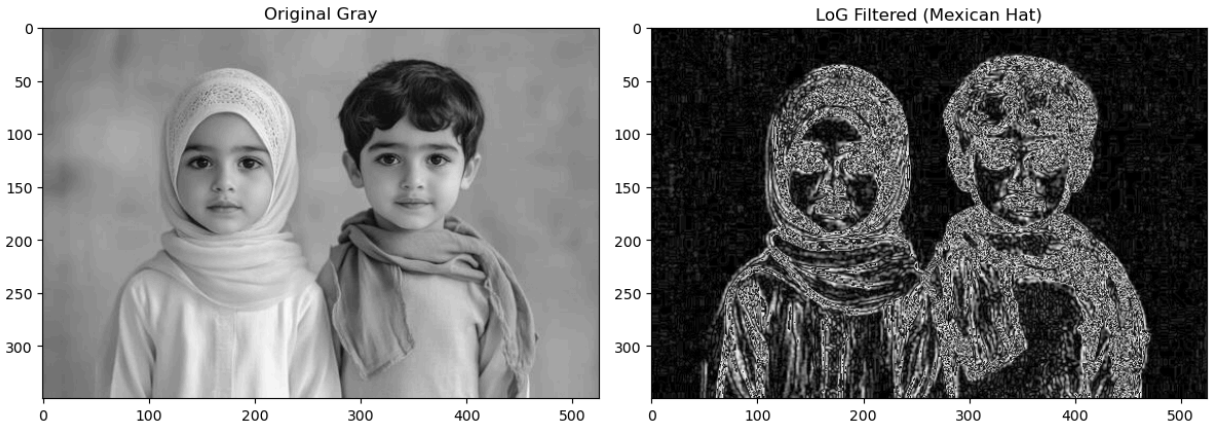
In [7]:
```
# 1. Load a real-world photo in grayscale
img = cv2.imread(imgFile, cv2.IMREAD_GRAYSCALE)

# 2. Step 1: Gaussian Blur (Smoothing)
# Larger kernel/sigma reduces more noise but may miss small edges
blurred = cv2.GaussianBlur(img, (5, 5), 0)

# 3. Step 2: Laplacian Operator (Second Derivative)
# cv2.CV_64F is used to store negative values (crucial for zero-crossings)
laplacian = cv2.Laplacian(blurred, cv2.CV_64F, ksize=5)

# 4. Post-processing: Normalize for display
# Since the Laplacian has negative and positive values, we take the absolute
log_final = np.uint8(np.absolute(laplacian))

# Visualization
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1), plt.imshow(img, cmap='gray'), plt.title('Original Gray
plt.subplot(1, 2, 2), plt.imshow(log_final, cmap='gray'), plt.title('LoG Fil
plt.tight_layout()
plt.show()
```



DoG Filter

In [8]:
```
# 1. Load the real-world photo
img = cv2.imread(imgFile, cv2.IMREAD_GRAYSCALE)

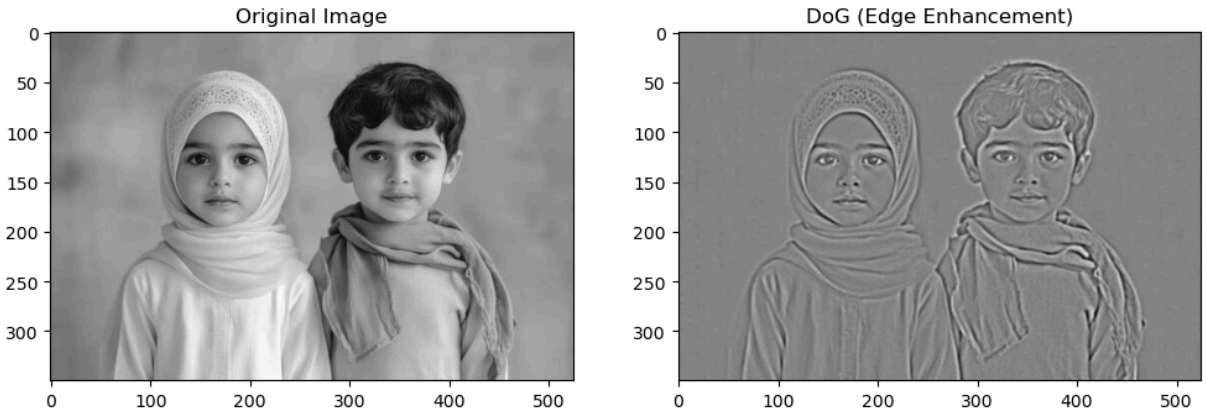
# 2. Apply two Gaussian blurs with different standard deviations (Sigmas)
# Low sigma preserves detail; High sigma captures broad structure
blur_narrow = cv2.GaussianBlur(img, (0, 0), sigmaX=1.0)
blur_wide = cv2.GaussianBlur(img, (0, 0), sigmaX=2.0)

# 3. Calculate the Difference (DoG)
# Note: We use float32 to avoid overflow/underflow during subtraction
dog = blur_narrow.astype(np.float32) - blur_wide.astype(np.float32)

# 4. Normalize or Threshold for visibility
# Option A: Map to 0-255 range
dog_normalized = cv2.normalize(dog, None, 0, 255, cv2.NORM_MINMAX).astype(np
```

```
# Option B: Simple threshold to create a binary edge map
_, dog_binary = cv2.threshold(dog_normalized, 127, 255, cv2.THRESH_BINARY)

# Visualization
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1), plt.imshow(img, cmap='gray'), plt.title('Original Imag
plt.subplot(1, 2, 2), plt.imshow(dog_normalized, cmap='gray'), plt.title('Do
plt.show()
```



Gradient Edge Detector for Colored Images

In [9]:

```
# 1. Load the real-world color image
image = cv2.imread(imgFile)

# 2. Split the image into its R, G, and B components
# Note: OpenCV uses BGR order by default
b, g, r = cv2.split(image)

def get_channel_gradient(channel):
    # Use Sobel for gradient calculation on a single channel
    # Blur first to reduce noise in that specific color layer
    blurred = cv2.GaussianBlur(channel, (3, 3), 0)
    gx = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
    gy = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)
    return cv2.magnitude(gx, gy)

# 3. Compute gradients for each color channel
grad_b = get_channel_gradient(b)
grad_g = get_channel_gradient(g)
grad_r = get_channel_gradient(r)

# 4. Combine gradients from all channels
# This ensures edges from color-only transitions are captured
combined_gradient = np.sqrt(grad_b**2 + grad_g**2 + grad_r**2)

# 5. Normalize to 0-255 for visualization
color_edges = cv2.normalize(combined_gradient, None, 0, 255, cv2.NORM_MINMAX

# Visualization
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1), plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB)), pl
```

```
plt.subplot(1, 2, 2), plt.imshow(color_edges, cmap='inferno'), plt.title('Co
plt.tight_layout()
plt.show()
```



Task 1: Apply Edge Detection Methods

A new image (`Butterfly.jpg`) is used instead of the one in the original notebook. The Sobel, Prewitt and Canny detectors are applied to it, following the same steps as above.

In [10]:
```
myImgFile = 'Butterfly.jpg'

img = cv2.imread(myImgFile)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

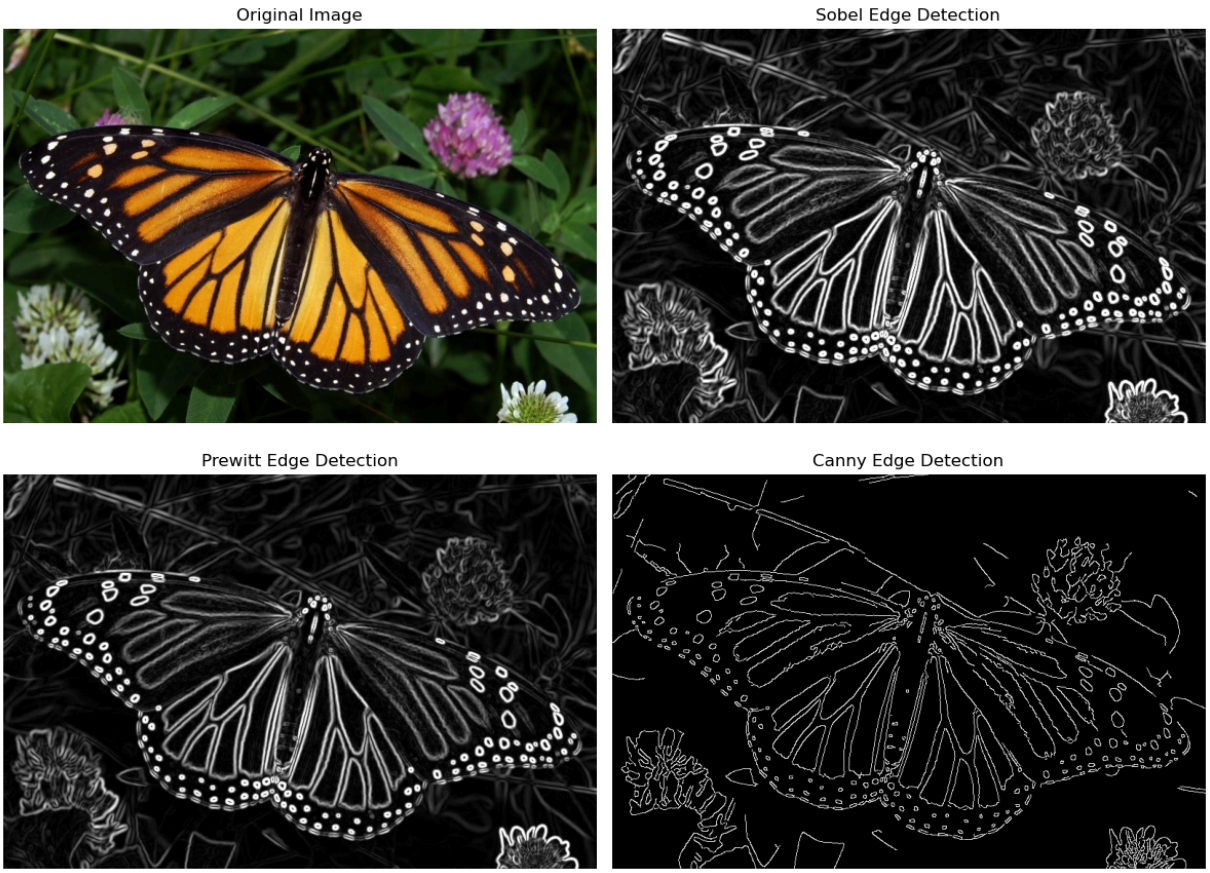
# Sobel
sobelx = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)
sobel_final = cv2.convertScaleAbs(cv2.magnitude(sobelx, sobely))

# Prewitt
kernel_x = np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]], dtype=np.float64)
kernel_y = np.array([[-1, -1, -1], [0, 0, 0], [1, 1, 1]], dtype=np.float64)
prewitt_x = cv2.filter2D(blurred, cv2.CV_64F, kernel_x)
prewitt_y = cv2.filter2D(blurred, cv2.CV_64F, kernel_y)
prewitt_final = cv2.convertScaleAbs(cv2.magnitude(prewitt_x, prewitt_y))

# Canny
canny_edges = cv2.Canny(blurred, 50, 150)

plt.figure(figsize=(12, 9))
plt.subplot(221), plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)), plt.titl
plt.subplot(222), plt.imshow(sobel_final, cmap='gray'), plt.title('Sobel Edg
plt.subplot(223), plt.imshow(prewitt_final, cmap='gray'), plt.title('Prewitt
plt.subplot(224), plt.imshow(canny_edges, cmap='gray'), plt.title('Canny Edg
for ax in plt.gcf().axes: ax.axis('off')
```

```
plt.tight_layout()
plt.show()
```

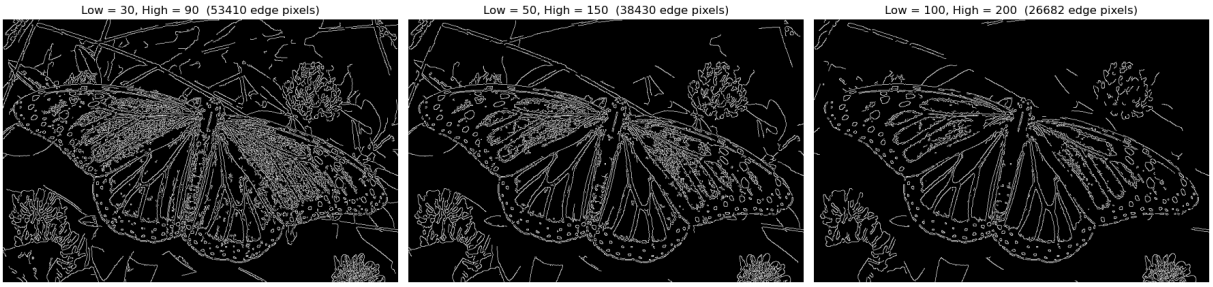


Sobel and Prewitt give a gradient magnitude map: edges are shown as thick grey lines whose brightness depends on how strong the change in intensity is. Sobel gives slightly stronger and smoother edges than Prewitt because its kernel puts more weight on the center row/column. Canny gives a binary map with thin, one pixel wide edges, because it uses non-maximum suppression and hysteresis thresholding.

Task 2: Effect of Canny Thresholds

In [11]:
```
thresholds = [(30, 90), (50, 150), (100, 200)]

plt.figure(figsize=(18, 5))
for i, (low, high) in enumerate(thresholds):
    edges = cv2.Canny(gray, low, high)
    plt.subplot(1, 3, i + 1), plt.imshow(edges, cmap='gray')
    plt.title(f'Low = {low}, High = {high}  ({np.count_nonzero(edges)} edge
    plt.axis('off')
plt.tight_layout()
plt.show()
```



Explanation

Canny uses two thresholds. A pixel with gradient above **High** is a strong edge, a pixel below **Low** is removed, and a pixel between them is only kept if it is connected to a strong edge.

- **Low = 30, High = 90:** the thresholds are small, so many weak gradients pass. This gives the largest number of edges and shows fine details (wing texture, veins, background), but also more noise and small unwanted edges.
- **Low = 50, High = 150:** a balance. The main outlines and the important wing patterns stay, while most of the small texture and noise are removed.
- **Low = 100, High = 200:** only strong gradients pass, so the number of edges is the lowest. The main contours of the butterfly are kept, but many fine details disappear and some edges become broken.

So increasing the thresholds reduces the number of detected edges and the visibility of fine details, and decreasing them shows more details but also more noise.